method that calls five other five-line methods, each of which does one thing and each of which has a name that is meaningful in the context of the application domain.

Prescription 14	Try to extract methods when you see compound Booleans, local variables, or inline comments.

Combine Duplication

You need to look out for three kinds of duplication: duplication of fact, duplication of logic, and duplication of structure.

Duplication of fact is usually easy to see and fix. A common case would be a "magic number" used by multiple parts of the code, such as a conversion factor or a maximum value. You saw this in an earlier example with the 21 days that you used to calculate velocity. Another common example is a status variable that has only a few valid values, and the list of those values is duplicated:

```
validates :size, numericality: {less_than_or_equal_to: 5}

def possible_sizes
  (1 .. 5)
end
```

The remedy for duplication of fact is also usually simple—make the value a constant or a method with a constant return value, like this:

```
MAX_POINT_COUNT = 5

validates :size, numericality: {less_than_or_equal_to: MAX_POINT_COUNT}

def possible_sizes
  (1 .. MAX_POINT_COUNT)
 end
```

Or, alternatively, like this:

```
VALID_POINT_RANGE = 1 .. 5
validates :size, inclusion: {in: VALID_POINT_RANGE}
```

The key metric is how many places in the code would need to change if the underlying facts change, with the ideal number being 1. That said, at some point the extra character count for a constant is ridiculous and Java-like in the worst way. For string and symbol constants, if the constant value is effectively identical to the symbol (as in ACTIVE_STATUS = :active), I'll often leave the duplication. I'm not saying I recommend that; I'm just saying I do it.

I also often make the constant value an instance method with a static return value rather than a Ruby constant, like this: